

Gestión Ágil con GitHub Projects & Kanban

Flujo de trabajo profesional: Del requisito (Issue) al despliegue (Pull Request)

Objetivo: Aprender a organizar, estimar y ejecutar el desarrollo de software en equipo aplicando la metodología Kanban de forma automatizada e integrada con el repositorio de código.

1. El Flujo de Trabajo en el Tablero

Columna	Descripción y Regla de Negocio	Límite WIP Recomendado
Todo (Por hacer)	Tareas priorizadas y descritas listas para ser abordadas.	Sin límite estricto
In Progress (En curso)	Tareas que se están desarrollando activamente en este momento.	Máx. 1-2 por persona
In Review (En revisión)	Pull Request abierta esperando revisión de código (*Code Review*).	Máx. 3 en total
Done (Completado)	Código mergeado en <code>main</code> , tests pasando e Issue cerrada.	—

2. Pasos de Configuración y Uso

1 Crear el Tablero en GitHub

- Dirígete a la pestaña **Projects** dentro de tu organización o repositorio.
- Haz clic en **New project**.
- Selecciona la vista de plantilla **Board** y nómbralo con el nombre del proyecto o módulo formativo.
- Asegúrate de que las columnas coinciden con el flujo estándar (*Todo, In Progress, In Review, Done*).

2 Definir Tareas con Issues

Cada tarea debe ser un **Issue** específico, medible y acotado:

- **Título claro:** En imperativo o indicativo directo (ej. «*Implementar autenticación con JWT*» o «*Diseñar vista de Login en XML/Compose*»).
- **Cuerpo descriptivo:** Añade criterios de aceptación y listas de verificación con sintaxis Markdown:
 - [] Crear endpoint POST /login
 - [] Encriptar clave en BD con BCrypt
- **Asignación y Metadatos:** Asigna el miembro responsable (*Assignee*), añade etiquetas (*Labels*: `backend`, `feature`) y vincúlalo en el panel lateral a tu **Project**.

3 Ciclo de Desarrollo en Git

Nunca programes directamente sobre la rama principal. Sigue este ciclo:

1. Arrastra la tarjeta del Issue a la columna **In Progress**.
2. Crea una rama específica desde `main` nombrada según la tarea:

```
git checkout -b feature/issue-12-auth-jwt
```
3. Desarrolla la solución, realiza commits limpios y súbela al remoto:

```
git push origin feature/issue-12-auth-jwt
```

4 Creación de Pull Request y Vinculación Automática

Al terminar la tarea, abre una **Pull Request (PR)** hacia la rama `main`:

- En la descripción de la PR, incluye la palabra clave vinculante seguida del número de Issue:
`Closes #12` o `Fixes #12`
- Asigna a un compañero como **Reviewer** para validar el código.
- La tarjeta debe pasar a **In Review** mientras se inspecciona.

⚡ Automatización Clave

Cuando el revisor aprueba la PR y se ejecuta el **Merge pull request**, GitHub cerrará automáticamente el Issue vinculado (#12) y moverá la tarjeta al estado **Done** sin intervención manual.

5 Configurar Límite de Trabajo en Progreso (WIP Limit)

Para evitar la sobrecarga y promover la finalización de tareas:

1. En el tablero, pulsa en el menú de tres puntos (`...`) de la columna **In Progress**.
2. Selecciona **Set column limit** e introduce un valor (ej. `2` por desarrollador activo).
3. Si el equipo supera ese número de tareas activas, la cabecera se tornará de color naranja de alerta para exigir finalizar tareas antes de abrir nuevas.

3. Rúbrica de Verificación para el Alumno

- ☐ **Planificación:** Se han redactado los Issues con criterios claros antes de tocar una línea de código.
- ☐ **Asignación:** Ningún Issue en progreso carece de responsable asignado.
- ☐ **Flujo Git:** Existe correspondencia 1:1 entre rama de Git y número de Issue.
- ☐ **Trazabilidad:** La Pull Request contiene la cláusula `Closes #N` en su descripción.
- ☐ **Revisión:** El código no se mergea a `main` sin la aprobación explícita de al menos un compañero.